Buscaminas implementado en MARIE.js

Vaishali Desai (00335123), Andrés Lanchimba (00335631), Paula Caicedo (00335247)

ABSTRACT Este informe presenta el diseño e implementación de un juego de Buscaminas en lenguaje ensamblador MARIE, con un algoritmo de flood fill para la propagación automática de celdas vacías. El juego opera sobre una pantalla de 16×16 mediante dos arreglos paralelos que codifican tanto el estado lógico como visual del tablero. El flood fill se estructura en tres fases: una siembra vertical sobre la columna de la celda seleccionada, seguida de una propagación por ondas sobre el tablero completo. La mecánica del juego incluye detección de minas, marcado de banderas, seguimiento de la condición de victoria y una secuencia de game over. Todo el diseño está condicionado por las restricciones de MARIE, incluyendo la ausencia de multiplicación nativa, la imposibilidad de usar estructuras de datos dinámicas, y un modelo de subrutinas no anidable.

INDEX TERMS MARIE, ensamblador, arquitectura, simulación, buscaminas

I. INTRODUCCIÓN

El presente proyecto consiste en la implementación de una versión funcional del juego Buscaminas sobre el simulador MARIE.js, utilizando su display gráfico de 16×16 celdas como interfaz visual. El objetivo principal es desarrollar la lógica completa del juego en lenguaje ensamblador MARIE, respetando las limitaciones inherentes a esta arquitectura educativa de conjunto de instrucciones reducido.

Dado que MARIE no dispone de dispositivos de entrada interactivos como mouse o teclado, la interacción con el usuario se realiza mediante el ingreso secuencial de valores numéricos a través de la instrucción INPUT. En cada turno, el usuario ingresa tres valores: fila, columna y acción a ejecutar sobre la celda seleccionada.

Adicionalmente, la arquitectura no ofrece mecanismos prácticos para la generación de números aleatorios ni para la construcción dinámica de tableros. Por este motivo, el tablero inicial es generado externamente mediante un script en Python, el cual produce el contenido de memoria necesario para inicializar el estado del juego en MARIE. Este script también permite visualizar el tablero completo, facilitando la verificación de la lógica implementada.

II. METODOLOGÍA

La metodología del proyecto se divide en dos componentes. Primero, un script en Python que genera el tablero inicial y produce el bloque de datos de memoria necesario para inicializarlo en MARIE. Segundo, la implementación de la lógica completa del juego en ensamblador MARIE, abarcando el manejo de turnos, detección de minas, sistema de banderas y condición de victoria.

A. PYTHON

1) Generar Tablero

El tablero se genera mediante tres funciones:

- `generar_tablero` inicializa una cuadrícula de 16×16 en ceros y coloca las minas en posiciones aleatorias usando `random.sample`. La cantidad de minas es ingresada por el usuario en el rango 1-255, garantizando al menos una celda segura para poder ganar y al menos una mina para poder perder.
- `calcula_numeros` recorre todas las celdas no-mina y cuenta las minas adyacentes en su vecindad de 3×3, asignando el resultado como valor de la celda.
- Finalmente, `contar_palabras` parsea el archivo `.txt` de MARIE para determinar el offset de instrucciones antes del bloque de datos del tablero, permitiendo anclar correctamente los valores generados en la dirección de memoria correspondiente.

```
function generar_tablero():
    tablero <- matriz de 0s de 16x16
    posiciones <- lista de todas las celdas (fila, col)
    colocar MINA_VAL en 40 posiciones aleatorias sin repeticion
    return tablero

function calcular_numeros(tablero):
    for cada celda (f, c) que no sea mina:
```

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

```
                cuenta <- numero de minas en vecindad 3x3 de (f, c)
                tablero[f][c] <- cuenta
            return tablero

        function contar_palabras(lineas):
            count <- 0
            for cada linea en lineas:
                ignorar lineas vacias y comentarios
                extraer codigo util (ignorar inline comments)
                if linea tiene contenido: count ++
            return count
```

Listing 1. **Pseudocódigo: Generación del Tablero**

2) Indexar tablero

Este bloque anexa el tablero generado al archivo de código MARIE como una secuencia de directivas `DEC`. Cada celda se escribe como `DEC <valor>`, donde -1 representa una mina y $0-8$ la cantidad de minas adyacentes.

Se incluyen comentarios por fila con una representación visual del tablero (`*` para minas, número para el resto), facilitando la verificación manual. El resultado queda anclado en la dirección `0x400`, correspondiente a TablBase.

B. MARIE.js

1) Fase 1: Pintar todo el display

- `PintaG` se encarga de recorrer todas las posiciones de memoria del display y pintarlas de gris para empezar el juego.
- `DecrG` se encarga de disminuir el contador para que una vez pintado todo el display se inicie el juego.

2) Diseño de memoria

- Dos arreglos paralelos de 256 palabras: `TablBase` en `0x300` (estado lógico) y `DispBase` en `0xF00` (display visual).
- Codificación de estado directamente en los valores: `val` original = oculta, `val+100` = descubierta, `val+200` = bandera, `-1` = mina.
- Padding explícito para anclar `TablBase` exactamente en `0x300`.

3) Lógica del turno

- Cálculo del índice plano `idx = fila*16 + col` mediante 16 instrucciones `Add` consecutivas, dado que MARIE no dispone de instrucción de multiplicación nativa.
- Detección del estado de la celda por color del display: `Gris` indica oculta, `Amarillo` indica bandera, cualquier otro color indica celda ya revelada.
- Sistema de banderas: toggle basado en comparación de color, con bloqueo de revelado si hay bandera activa.

4) Revelado y condiciones de fin

- Detección de mina mediante `val + 1 == 0`, aprovechando la representación en complemento a dos.
- Game over (`RevMin`): barrido de las 256 celdas pintando todas las minas en negro seguido de `Halt`.
- Condición de victoria: contador `CntSeg` inicializado en 246 ($256 - 10$ minas); al llegar a 0 se ejecuta `Halt`.
- Despacho de colores (`PintaColor`): subrutina compartida con cascada `Subt` + `Skipcond 400` para valores 0–8.

5) Flood fill

- Fases 1 y 2: siembra vertical (`SubirLoop` / `BajarLoop`) sobre la columna de la celda seleccionada, revelando el tramo continuo de celdas vacías antes de iniciar la propagación.
- Fase 3: propagación por ondas (`FFWLOOP`) con bucle externo por columnas y control de cambios mediante `FFChg`.
- Subrutina `FFRVNGH`: verificación de bounds por signo en complemento a dos, detección de mina, y llamada anidada a `PintaColor` via `JnS`.

```
/ -- Calculo de indice plano --
// 16 Add consecutivos
idx <- fila + fila + ... (x16) + col

/ -- Deteccion de estado por color del display --
color <- display[idx]
// oculta sin bandera
if color == Gris:    goto ChkAccion

// tiene bandera
if color == Amarillo: goto ChkBandera

// ya revelada, ignorar
else: goto Turno

/ -- Deteccion de mina --
if tablero[idx] + 1 == 0: goto RevMin

/ -- Game over --
for i in 0..255:
    if tablero[i] == -1 or tablero[i] == MinaBand:
        display[i] <- Negro
```

```
Halt

/ -- Flood fill: siembra vertical --
SubirLoop:
    CurIdx <- CurIdx - 16
    if CurIdx < 0: stop
    if display[CurIdx] != Gris: stop
    if tablero[CurIdx] == mina: stop
    revelar celda CurIdx
    if tablero[CurIdx] != 0: stop      // era numero, no seguir subiendo
    goto SubirLoop

// simetrico hacia abajo
BajarLoop:
    CurIdx <- CurIdx + 16
    if CurIdx >= 256: stop
    // mismas condiciones de parada
    goto BajarLoop

/ -- Flood fill: propagacion por ondas --
FFChg <- 1
while FFChg > 0:
    FFChg <- 0
    for WC in 0..15:
        for WR in 0..15:
            if display[WR*16+WC] == GrisC:
                for each vecino (NR, NC) in vecindad 8:
                    call FFRVNGH

/ -- Subrutina FFRVNGH --
if NR < 0 or NR >= 16 or NC < 0 or NC >= 16: return
NIDX <- NR*16 + NC
if display[NIDX] != Gris: return
if tablero[NIDX] == mina: return
call PintaColor(WVal, WDPtr)
tablero[NIDX] <- tablero[NIDX] + 100
CntSeg <- CntSeg - 1
FFChg <- 1
return
```

Listing 2. **Pseudocódigo: Lógica principal del juego en MARIE**

III. RESULTADOS

Ejemplo de ejecución

La Figura 1 muestra el estado del display de MARIE tras una partida terminada en derrota. El tablero se encuentra casi completamente revelado, con las celdas descubiertas mostrando su color correspondiente según la cantidad de minas adyacentes: gris claro para celdas vacías, y azul, verde y demás colores para valores del 1 al 8. Al haber clickeado una mina, se ejecutó la secuencia de game over, revelando automáticamente todas las minas restantes en color negro.

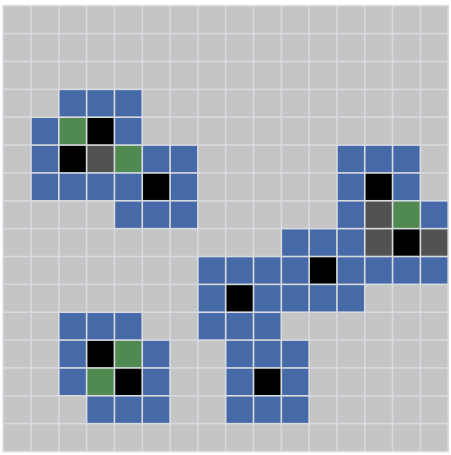

FIGURE 1. **Ejemplo de ejecución del programa.**

IV. CONCLUSIONES

El desarrollo de Buscaminas en ensamblador MARIE demostró que es posible implementar una lógica de juego completa bajo restricciones arquitectónicas severas, siempre que el diseño se adapte a ellas desde el inicio. La codificación de estado directamente en los arreglos existentes, el cálculo de índices mediante adiciones repetidas, y la detección de condiciones vía complemento a dos son ejemplos concretos de cómo las limitaciones del conjunto de instrucciones obligan a soluciones que explotan la representación interna de los datos. En cuanto al flood fill, la propagación por ondas resultó ser la única alternativa viable dado que MARIE no soporta estructuras de datos dinámicas. En conjunto, el proyecto evidencia que el diseño en bajo nivel no consiste únicamente en traducir algoritmos existentes, sino en repensar su estructura en función de lo que la arquitectura permite expresar.

REFERENCIAS

Referencias

Elijah Flader. (2023, March 16). Building minesweeper in Python. Medium. https://medium.com/@elijahflader/how-to-make-minesweeper-python-16c71390206e

Minesweeper Online. (n.d.). Minesweeper online. https://minesweeper.online/

Sheppard, M. (2023, October 9). A simple minesweeper algorithm. https://minesweepergame.com/math/a-simple-minesweeper-algorithm-2023.pdf